PROGRAMAÇÃO PARA CIÊNCIA DE DADOS · 2026.1

# Pandas

A tabela inteira, com nome nas colunas
— 6.433 corridas de táxi de Nova York

AULA 04 • PYTHON

EM UMA FRASE

# O que é

PANDAS

É a **planilha inteira** dentro do Python: número, texto e data na mesma tabela.

**Uma tabela só**
as colunas andam juntas — filtrar uma arrasta a linha inteira

**Nome, não posição**
você escreve `df["fare"]`, e não a coluna número cinco

TRÊS COISAS

# Para que serve

```python
import pandas as pd

df = pd.read_csv("dados/taxis.csv")    # df = a tabela
```

📂

**Ler**

CSV, Excel, banco de dados

```python
pd.read_csv(...)
```

🔍

**Olhar**

o resumo da tabela inteira

```python
df.describe()
```



**Agrupar**

uma resposta por categoria

```python
df.groupby("payment")
```

DE ONDE VEM A TABELA

# Carregar

```
pd.read_csv("taxis.csv")          # arquivo de texto — o desta aula
pd.read_excel("vendas.xlsx")      # planilha do Excel
pd.read_sql(consulta, conexao)    # banco de dados da empresa
pd.read_json(resposta)            # o que uma API devolveu
pd.read_clipboard()               # o que você acabou de copiar
pd.DataFrame(dados)               # dicionário ou lista, na mão
```

---

E ainda `parquet`, `html`, `xml`, `sas`, `spss`... a lista é longa.

> Toda fonte entra pela mesma porta — um `pd.read_…` e sai um DataFrame. Muda a primeira linha; o resto da análise é o mesmo.

READ_CSV

# CSV

O ARQUIVO, ABERTO NO BLOCO DE NOTAS

```
distance,fare,payment
1.6,7.0,credit card
0.79,5.0,cash
```

O DATAFRAME

| | DISTANCE | FARE | PAYMENT |
|---|---|---|---|
| 0 | 1.60 | 7.00 | credit card |
| 1 | 0.79 | 5.00 | cash |

A **primeira linha** do arquivo virou o nome das colunas. O `0` e o `1` à esquerda são o **índice**, que o Pandas põe sozinho.

READ_EXCEL

# Excel

A PLANILHA

| | A | B |
|---|---|---|
| 1 | loja | valor |
| 2 | Centro | 12500 |
| 3 | Shopping | 31800 |

O DATAFRAME

| | LOJA | VALOR |
|---|---|---|
| 0 | Centro | 12500 |
| 1 | Shopping | 31800 |

A linha **1** virou cabeçalho. E a contagem recomeça no **0**: Excel conta de 1, Pandas de 0.

Outra aba: `sheet_name="2026"`. Título em cima: `skiprows=3`.

READ_SQL

# Banco de dados

A CONEXÃO E A CONSULTA

```
conexao = create_engine(
    "postgresql://usuario:senha@host/loja")

consulta = "select loja, valor from vendas"
df = pd.read_sql(consulta, conexao)
```

O DATAFRAME

| | LOJA | VALOR |
|---|---|---|
| 0 | Centro | 12500 |
| 1 | Shopping | 31800 |

As linhas que o banco devolve chegam **já como tabela**. O filtro pesado fica no `select`: traga só as colunas e o período que você vai usar.

READ_JSON

# JSON

**O QUE A API DEVOLVEU**

```
[
  {"loja":"Centro","valor":12500},
  {"loja":"Shopping","valor":31800}
]
```

**O DATAFRAME**

| | LOJA | VALOR |
|---|---|---|
| 0 | Centro | 12500 |
| 1 | Shopping | 31800 |

Cada objeto `{...}` é uma **linha**; cada chave, uma **coluna**. Quando vem aninhado, `json_normalize` abre os níveis em colunas.

DATAFRAME

# Na mão

O DICIONÁRIO QUE VOCÊ DIGITOU

```
dados = {
    "loja":  ["Centro", "Shopping"],
    "valor": [12500, 31800],
}
```

O DATAFRAME

| | LOJA | VALOR |
|---|---|---|
| 0 | Centro | 12500 |
| 1 | Shopping | 31800 |

**Chave é o nome da coluna, lista são os valores.** É como se monta um exemplo de dez linhas para testar uma ideia sem depender de arquivo.



# Na prática

Um método por slide: o comando, o que ele devolve, o que isso quer dizer

SHAPE · HEAD

# A tabela

```
>>> df.shape
(6433, 14)
>>> df[["distance", "fare", "tip", "payment"]].head(3)
   distance  fare   tip      payment
0      1.60   7.0  2.15  credit card
1      0.79   5.0  0.00         cash
2      1.37   7.5  2.36  credit card
```

O `shape` dá o tamanho; o `head` mostra o começo. Quatro das catorze colunas aparecem aqui.

INFO

# info

```
>>> df[["distance", "fare", "payment"]].info()
<class 'pandas.DataFrame'>
RangeIndex: 6433 entries, 0 to 6432
Data columns (total 3 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   distance  6433 non-null   float64
 1   fare      6433 non-null   float64
 2   payment   6389 non-null   str
dtypes: float64(2), str(1)
memory usage: 207.9 KB
```

Uma linha responde "o que eu tenho na mão?": quantas linhas, o tipo de cada coluna e **quantos valores existem** — repare no 6389.

DESCRIBE

# describe

```
>>> df[["fare", "tip"]].describe()
              fare         tip
count  6433.000000  6433.00000  # quantas linhas têm valor
mean     13.091073     1.97922  # a média
std      11.551804     2.44856  # o quanto os valores se espalham
min       1.000000     0.00000  # o menor de todos
25%       6.500000     0.00000  # 1 em cada 4 fica abaixo daqui
50%       9.500000     1.70000  # a mediana — metade abaixo, metade acima
75%      15.000000     2.80000  # 3 em cada 4 ficam abaixo daqui
max     150.000000    33.20000  # o maior de todos
```

Oito números por coluna de uma vez. A tarifa tem média **13,09** e mediana **9,50**: metade das corridas custa menos de dez dólares.

SERIES · TO_NUMPY

# Uma coluna

```
>>> df["fare"].head(3)
0    7.0
1    5.0
2    7.5
Name: fare, dtype: float64
>>> print(df["fare"].mean())
13.091072594434944
>>> df["fare"].to_numpy()
array([ 7. ,  5. ,  7.5, ..., 16. ,  6. , 15. ], shape=(6433,))
```

Uma coluna é uma **Series**: valores, índice e nome. O `to_numpy()` descarta o índice e devolve só os números.

MÁSCARA

# Filtrar

```
>>> longas = df[df["distance"] > 10]
>>> longas.shape
(394, 14)
>>> print(longas["fare"].mean())
46.52796954314721
```

A condição entre colchetes devolve a tabela **inteira**, só com essas 394 linhas — as catorze colunas continuam ali.

MÁSCARA · QUERY

# Duas condições

```
>>> mascara = (df["distance"] > 10) & (df["tip"] > 0)  # o & pede as duas
>>> mascara.head(3)
0    False
1    False
2    False
dtype: bool
>>> df[mascara].shape  # só as linhas True
(225, 14)
>>> df.query("distance > 10 and tip > 0").shape  # aqui, "and"
(225, 14)
```

O `&` compara **linha por linha** e devolve uma coluna de True e False. O `and` do Python não serve na máscara — dentro do `query`, serve.

Os parênteses são obrigatórios: o `&` liga mais forte que o `>`.

VALUE_COUNTS

# value_counts

```
>>> df["payment"].value_counts(dropna=False)
payment
credit card    4577
cash           1812
NaN              44
Name: count, dtype: int64
>>> df["distance"].nunique()
1079
```

Serve para **categoria**: texto, ou número de poucos valores como `passengers`. Em número contínuo não ajuda — a distância tem 1.079 valores diferentes.

Sem o `dropna=False`, a linha do `NaN` não aparece. Para número contínuo, `describe()`.

GROUPBY

# groupby

```
>>> df.groupby("payment")["tip"].mean()
payment
cash           0.000000
credit card    2.781805
Name: tip, dtype: float64
```

**Separe por, e calcule.** Quem paga em dinheiro não deixa gorjeta registrada; no cartão, são 2,78 em média.



# Com gráficos

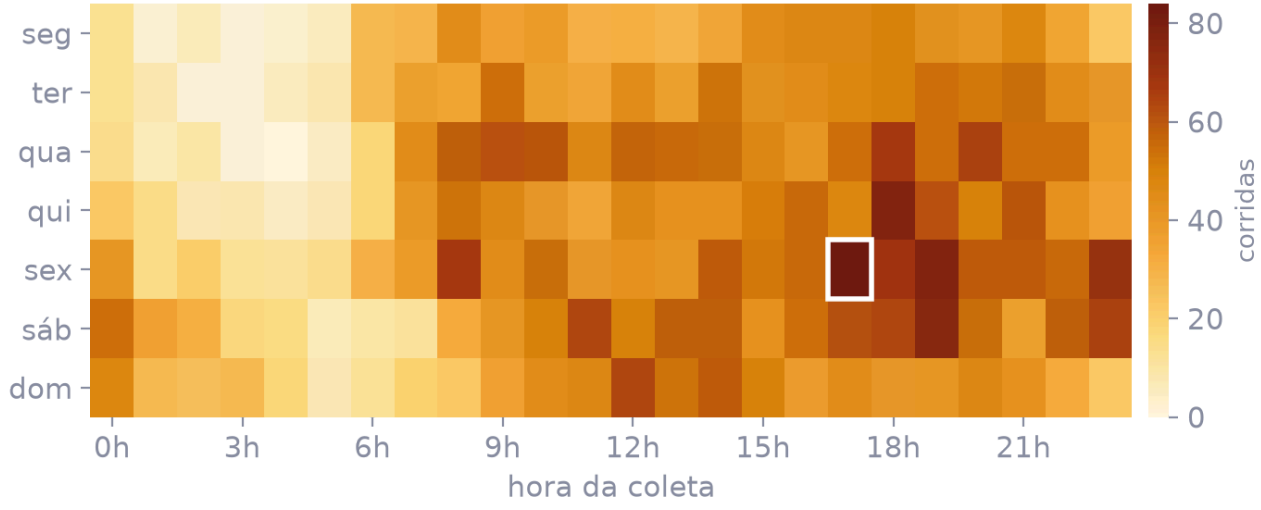
Uma pergunta por slide — e o desenho da tabela que responde

MAPA DE CALOR · PIVOT_TABLE

# Quando as pessoas pegam táxi?

```
mapa = df.pivot_table(index=df["pickup"].dt.day_name(),
                      columns=df["pickup"].dt.hour, aggfunc="size")

plt.imshow(mapa)     # 7 linhas × 24 colunas de contagem
```




O pico é **sexta, 17h**: 84 corridas. E a madrugada só tem movimento no **fim de semana** — dia útil às 4h, quase nada.

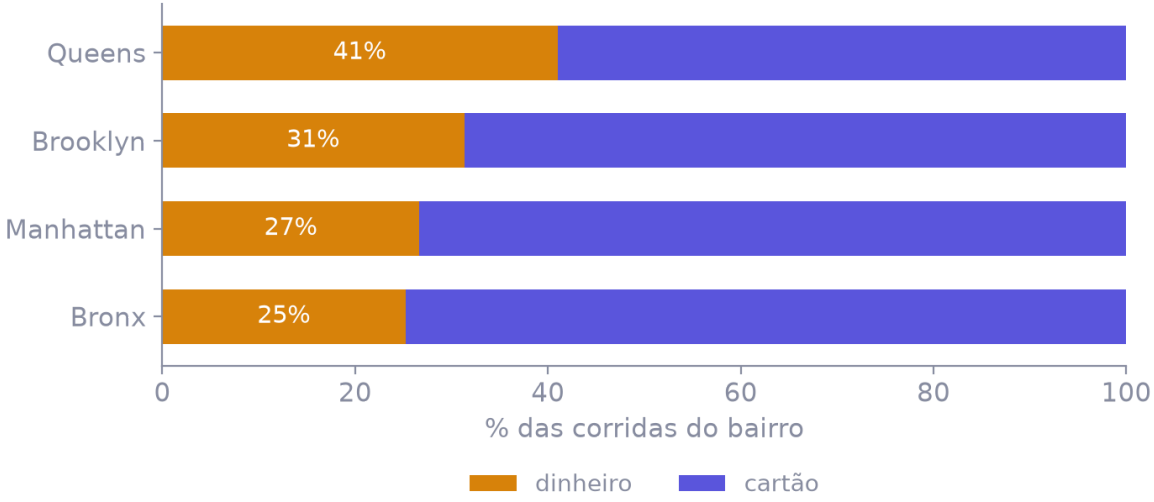
**Uma coluna só:** `pickup`. Dela saem o dia e a hora; o `size` conta as linhas de cada cruzamento.

BARRAS EMPILHADAS · UNSTACK E DIV

# Onde ainda se paga em dinheiro?

```
qtd = df.groupby(["pickup_borough", "payment"]).size().unstack()

pct = qtd.div(qtd.sum(axis=1), axis=0) * 100   # cada linha vale 100
pct.plot(kind="barh", stacked=True)
```




No **Queens**, 41% pagam em dinheiro; em **Manhattan**, 27%. O `div(axis=0)` divide cada linha pelo seu total — bairros de tamanhos diferentes na mesma régua.

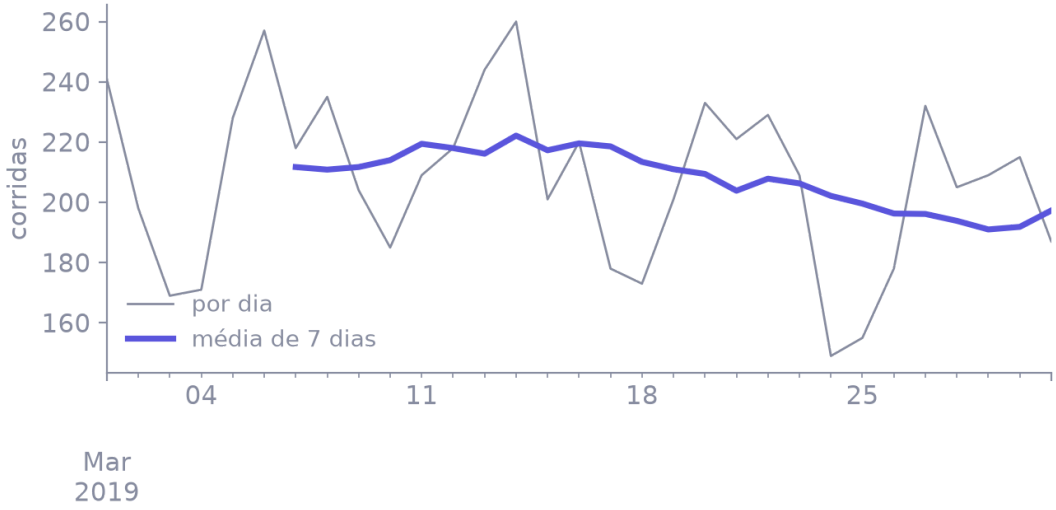
**Duas colunas, as duas de texto:** `pickup_borough` e `payment`.

GRÁFICO DE LINHAS · RESAMPLE E ROLLING

# O movimento muda ao longo do mês?

```
marco = df.set_index("pickup").loc["2019-03"]   # o mês pela string

por_dia = marco.resample("D").size()            # uma linha por dia
por_dia.rolling(7).mean().plot()                # a média de 7 dias
```




O serrote é a semana; a linha grossa é a **tendência**. Pico em **14/03** com 260 corridas, fundo em **24/03** com 149.

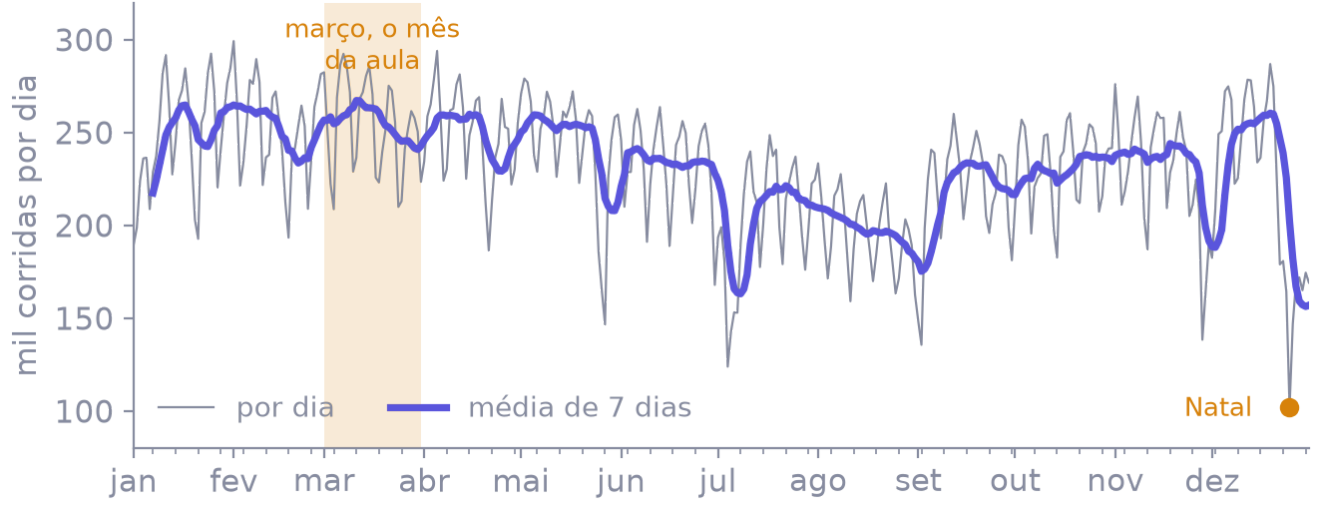
**De novo só o** `pickup` — desta vez como índice, para o `resample` agrupar por dia.

GRÁFICO DE LINHAS · 365 DIAS

# E no ano inteiro?

```
ano = pd.read_csv("corridas-2019.csv", parse_dates=["dia"], index_col="dia")

ano["corridas"].plot()                    # os 365 dias
ano["corridas"].rolling(7).mean().plot()  # a mesma média de 7 dias
```




**84,6 milhões** de corridas em 365 pontos. **Agosto** é o fundo do ano, **março** o topo — e cada mergulho é um feriado. O menor dia é o **Natal**.

**Outro arquivo, duas colunas:** `dia` e `corridas` — já resumidas, uma linha por dia.

FECHANDO

# Resumo

PANDAS

Serve para **visualizar**, **agrupar** e **destilar** — sempre na tabela inteira, sem laço.

🔍

**Visualizar**

abrir o arquivo e ver o que tem dentro — `info` · `describe`

🧱

**Agrupar**

uma resposta por categoria — `groupby` · `value_counts`

📈

**Destilar**

milhões de linhas viram poucos números — que cabem num gráfico